```
"""
STEP 1: Install and import required libraries

This step ensures that all necessary libraries for performing
machine translation using Hugging Face Transformers are available.

Libraries:
- transformers: Provides pretrained NLP models and pipelines
- torch: Backend deep learning framework used by transformers
- sentencepiece: Helps in tokenization for some translation models
"""

# Install required libraries (run only once)
!pip install transformers torch sentencepiece

"""
Import the pipeline function from transformers.

pipeline() is a high-level API that allows us to perform tasks like
translation, sentiment analysis, etc., with a single function call.
"""
from transformers import pipeline
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: sentencepiece in /usr/local/lib/python3.12/dist-packages (0.2.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.25.2)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (1.7.1)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.2)
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (1.4.2)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (0.28.1)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (0.24.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (4.12.1)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (2026.2.25)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (1.0.9)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (3.11)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.3.0->transformers) (8.3.1)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.3.0->transformers) (1.5.4)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.3.0->transformers) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.3.0->transformers) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-hub<2.0,>=1.3.0->transformers) (
```

```
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-hub<2.0,>=1.3.0->transformers)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer->huggingface-hub<2.0,>=1.3.0->tra
```

```
import torch
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

"""
STEP 2: Load translation models using Hugging Face AutoClasses

In this step, we explicitly load tokenizers and models for multiple languages:
- English → French
- English → German
- English → Hindi

This approach directly uses AutoTokenizer and AutoModelForSeq2SeqLM
to circumvent an issue with the pipeline function's task registration.
"""

# Determine device to use
device = 0 if torch.cuda.is_available() else -1

# Load English to French translation model and tokenizer
model_name_fr = "Helsinki-NLP/opus-mt-en-fr"
tokenizer_fr = AutoTokenizer.from_pretrained(model_name_fr)
model_fr = AutoModelForSeq2SeqLM.from_pretrained(model_name_fr).to(device if device != -1 else 'cpu')

# Load English to German translation model and tokenizer
model_name_de = "Helsinki-NLP/opus-mt-en-de"
tokenizer_de = AutoTokenizer.from_pretrained(model_name_de)
model_de = AutoModelForSeq2SeqLM.from_pretrained(model_name_de).to(device if device != -1 else 'cpu')

# Load English to Hindi translation model and tokenizer
model_name_hi = "Helsinki-NLP/opus-mt-en-hi"
tokenizer_hi = AutoTokenizer.from_pretrained(model_name_hi)
model_hi = AutoModelForSeq2SeqLM.from_pretrained(model_name_hi).to(device if device != -1 else 'cpu')

"""
Verification step:
Print a message to confirm that models are loaded successfully.
"""
print("All translation models and tokenizers loaded successfully!")
```

```
Loading weights: 100%                                   258/258 [00:00<00:00, 280.01it/s, Materializing param=model.shared.weight]
The tied weights mapping and config for this model specifies to tie model.shared.weight to model.decoder.embed_tokens.weight, but both are present in the checkpoints,
The tied weights mapping and config for this model specifies to tie model.shared.weight to model.encoder.embed_tokens.weight, but both are present in the checkpoints,
Loading weights: 100%                                   258/258 [00:00<00:00, 401.18it/s, Materializing param=model.shared.weight]
The tied weights mapping and config for this model specifies to tie model.shared.weight to model.decoder.embed_tokens.weight, but both are present in the checkpoints,
The tied weights mapping and config for this model specifies to tie model.shared.weight to model.encoder.embed_tokens.weight, but both are present in the checkpoints,
Loading weights: 100%                                   258/258 [00:00<00:00, 441.81it/s, Materializing param=model.shared.weight]
The tied weights mapping and config for this model specifies to tie model.shared.weight to model.decoder.embed_tokens.weight, but both are present in the checkpoints,
The tied weights mapping and config for this model specifies to tie model.shared.weight to model.encoder.embed_tokens.weight, but both are present in the checkpoints,
All translation models and tokenizers loaded successfully!
```

```
"""
STEP 3: Prepare dataset (input sentences)

We create a list of sentences containing different types:
- Simple sentences
- Compound sentences
- Complex sentences
- Interrogative sentences
- Technical sentences
- Conversational sentences

This diversity helps evaluate translation performance across
different linguistic structures.
"""

# List of input sentences
sentences = [
    "I am learning machine learning.",
    "She likes coffee and tea.",
    "Although it was raining, we went outside.",
    "What are you doing now?",
    "Artificial Intelligence is transforming industries.",
    "He went to the market and bought vegetables.",
    "Can you help me with this problem?",
    "The research paper was published in a journal.",
    "I will call you later.",
    "Do you understand this concept?",
    "We are planning a trip next month.",
    "The server crashed due to high traffic.",
    "She is reading a novel.",
    "Where do you live?",
    "This algorithm improves performance significantly."
]

"""
Display the number of sentences to confirm dataset preparation.
"""
print("Total sentences:", len(sentences))
```

```
Total sentences: 15
```

```
"""
STEP 4: Perform translation from English to French

For each sentence:
1. Pass it to the translation pipeline
2. Extract translated text
3. Print both original and translated sentence

This helps in observing translation quality.
"""

# Loop through each sentence
for sentence in sentences:
```

```
    # Tokenize the input sentence
    inputs = tokenizer_fr(sentence, return_tensors="pt", padding=True, truncation=True)

    # Perform translation
    with torch.no_grad():
        translated_tokens = model_fr.generate(
            input_ids=inputs["input_ids"],
            attention_mask=inputs["attention_mask"],
            max_length=512
        )

    # Decode the translated text
    translated_text = tokenizer_fr.decode(translated_tokens[0], skip_special_tokens=True)

    # Print outputs
    print("English:", sentence)
    print("French:", translated_text)
    print("-" * 50)  # Separator for readability
```

```
English: I am learning machine learning.
French: J'apprends l'apprentissage automatique.
--------------------------------------------------
English: She likes coffee and tea.
French: Elle aime le café et le thé.
--------------------------------------------------
English: Although it was raining, we went outside.
French: Même s'il pleuvait, nous sommes sortis.
--------------------------------------------------
English: What are you doing now?
French: Qu'est-ce que tu fais maintenant ?
--------------------------------------------------
English: Artificial Intelligence is transforming industries.
French: L'intelligence artificielle transforme les industries.
--------------------------------------------------
English: He went to the market and bought vegetables.
French: Il est allé au marché et a acheté des légumes.
--------------------------------------------------
English: Can you help me with this problem?
French: Pouvez-vous m'aider avec ce problème ?
--------------------------------------------------
English: The research paper was published in a journal.
French: Le document de recherche a été publié dans une revue.
--------------------------------------------------
English: I will call you later.
French: Je te rappelle plus tard.
--------------------------------------------------
English: Do you understand this concept?
French: Comprenez-vous ce concept ?
--------------------------------------------------
English: We are planning a trip next month.
French: Nous prévoyons un voyage le mois prochain.
--------------------------------------------------
English: The server crashed due to high traffic.
French: Le serveur s'est écrasé en raison d'un trafic élevé.
--------------------------------------------------
English: She is reading a novel.
French: Elle lit un roman.
--------------------------------------------------
English: Where do you live?
```

```
French: Où habitez-vous ?
--------------------------------------------------
English: This algorithm improves performance significantly.
French: Cet algorithme améliore considérablement les performances.
--------------------------------------------------
```

```
"""
STEP 5: Translate sentences into multiple languages

We translate each sentence into:
- French
- German
- Hindi

This allows us to compare how different languages handle:
- Grammar
- Sentence structure
- Vocabulary
"""

# Loop through sentences
for sentence in sentences:

    # Translate into French
    inputs_fr = tokenizer_fr(sentence, return_tensors="pt", padding=True, truncation=True)
    with torch.no_grad():
        translated_tokens_fr = model_fr.generate(
            input_ids=inputs_fr["input_ids"],
            attention_mask=inputs_fr["attention_mask"],
            max_length=512
        )
    fr = tokenizer_fr.decode(translated_tokens_fr[0], skip_special_tokens=True)

    # Translate into German
    inputs_de = tokenizer_de(sentence, return_tensors="pt", padding=True, truncation=True)
    with torch.no_grad():
        translated_tokens_de = model_de.generate(
            input_ids=inputs_de["input_ids"],
            attention_mask=inputs_de["attention_mask"],
            max_length=512
        )
    de = tokenizer_de.decode(translated_tokens_de[0], skip_special_tokens=True)

    # Translate into Hindi
    inputs_hi = tokenizer_hi(sentence, return_tensors="pt", padding=True, truncation=True)
    with torch.no_grad():
        translated_tokens_hi = model_hi.generate(
            input_ids=inputs_hi["input_ids"],
            attention_mask=inputs_hi["attention_mask"],
            max_length=512
        )
    hi = tokenizer_hi.decode(translated_tokens_hi[0], skip_special_tokens=True)

    # Print results
    print("English:", sentence)
    print("French :", fr)
```

```
    print("German :", de)
    print("Hindi  :", hi)
    print("=" * 60)  # Separator
```

```
English: I am learning machine learning.
French : J'apprends l'apprentissage automatique.
German : Ich lerne maschinelles Lernen.
Hindi  : मैं मशीन सीखने के लिए सीख रहा हूँ.
============================================================
English: She likes coffee and tea.
French : Elle aime le café et le thé.
German : Sie mag Kaffee und Tee.
Hindi  : वह कॉफी और चाय पसंद करती है.
============================================================
English: Although it was raining, we went outside.
French : Même s'il pleuvait, nous sommes sortis.
German : Obwohl es regnete, gingen wir nach draußen.
Hindi  : हालाँकि बारिश हो रही थी, फिर भी हम बाहर चले गए ।
============================================================
English: What are you doing now?
French : Qu'est-ce que tu fais maintenant ?
German : Was machst du jetzt?
Hindi  : अब तुम क्या कर रहे हो?
============================================================
English: Artificial Intelligence is transforming industries.
French : L'intelligence artificielle transforme les industries.
German : Künstliche Intelligenz verändert die Industrie.
Hindi  : कलात्मक तत्व उद्योगों को परिवर्तित कर रहा है.
============================================================
English: He went to the market and bought vegetables.
French : Il est allé au marché et a acheté des légumes.
German : Er ging auf den Markt und kaufte Gemüse.
Hindi  : वह बाजार गया और सब्ज़ियाँ खरीदा.
============================================================
English: Can you help me with this problem?
French : Pouvez-vous m'aider avec ce problème ?
German : Können Sie mir bei diesem Problem helfen?
Hindi  : आप इस समस्या के साथ मेरी मदद कर सकते हैं?
============================================================
English: The research paper was published in a journal.
French : Le document de recherche a été publié dans une revue.
German : Das Forschungspapier wurde in einer Zeitschrift veröffentlicht.
Hindi  : एक पत्रिका में शोध कागज प्रकाशित किया गया ।
============================================================
English: I will call you later.
French : Je te rappelle plus tard.
German : Ich rufe dich später an.
Hindi  : मैं तुम्हें बाद में फोन करेंगे.
============================================================
English: Do you understand this concept?
French : Comprenez-vous ce concept ?
German : Verstehen Sie dieses Konzept?
Hindi  : क्या आप इस धारणा को समझते हैं?
============================================================
English: We are planning a trip next month.
French : Nous prévoyons un voyage le mois prochain.
German : Wir planen eine Reise nächsten Monat.
Hindi  : हम अगले महीने यात्रा की योजना बना रहे हैं.
============================================================
English: The server crashed due to high traffic.
French : Le serveur s'est écrasé en raison d'un trafic élevé.
```

```
German : Der Server ist aufgrund des hohen Verkehrsaufkommens abgestürzt.
```

```
"""
STEP 6: Analysis of translation quality

In this section, we provide observations based on:
- Accuracy of meaning
- Grammar correctness
- Fluency
- Handling of technical terms

This is a textual analysis section (no computation required).
"""

analysis = """
The translation model performs well for simple and conversational sentences.
Most translations maintain correct grammar and structure across languages.
Technical terms such as 'algorithm' are sometimes retained without translation.
Complex sentences are translated with slight restructuring.
Hindi translations appear more natural compared to German in some cases.
Some contextual or cultural meanings are not fully preserved.
Interrogative sentences are translated correctly in most cases.
Fluency is generally high, but minor errors may occur.
Long sentences occasionally lose clarity.
Overall, pretrained models provide fast and efficient translation with good accuracy.
"""

# Print analysis
print(analysis)
```

```
The translation model performs well for simple and conversational sentences.
Most translations maintain correct grammar and structure across languages.
Technical terms such as 'algorithm' are sometimes retained without translation.
Complex sentences are translated with slight restructuring.
Hindi translations appear more natural compared to German in some cases.
Some contextual or cultural meanings are not fully preserved.
Interrogative sentences are translated correctly in most cases.
Fluency is generally high, but minor errors may occur.
Long sentences occasionally lose clarity.
Overall, pretrained models provide fast and efficient translation with good accuracy.
```

```
"""
STEP 7: Test translation on complex sentences

We evaluate model performance on:
- Long sentences
- Compound sentences
- Complex sentences
- Questions
- Domain-specific sentences

This helps identify limitations of the model.
"""
```

```
# Complex sentence list
complex_sentences = [
    "Although the system was designed efficiently, it failed due to unexpected errors.",
    "If you complete the assignment on time, you will receive full marks.",
    "The professor, who is an expert in AI, explained the concept clearly.",
    "What will you do if the model does not perform well?",
    "Machine learning models require large datasets to achieve high accuracy."
]

# Translate and print results
for sentence in complex_sentences:

    print("English:", sentence)

    # Translate into French
    inputs_fr = tokenizer_fr(sentence, return_tensors="pt", padding=True, truncation=True)
    with torch.no_grad():
        translated_tokens_fr = model_fr.generate(
            input_ids=inputs_fr["input_ids"],
            attention_mask=inputs_fr["attention_mask"],
            max_length=512
        )
    translated = tokenizer_fr.decode(translated_tokens_fr[0], skip_special_tokens=True)

    print("French:", translated)
    print("-" * 50)
```

```
English: Although the system was designed efficiently, it failed due to unexpected errors.
French: Bien que le système ait été conçu efficacement, il a échoué en raison d'erreurs inattendues.
--------------------------------------------------
English: If you complete the assignment on time, you will receive full marks.
French: Si vous remplissez l'affectation à temps, vous recevrez tous les points.
--------------------------------------------------
English: The professor, who is an expert in AI, explained the concept clearly.
French: Le professeur, qui est un expert en IA, a expliqué clairement le concept.
--------------------------------------------------
English: What will you do if the model does not perform well?
French: Que ferez-vous si le modèle ne fonctionne pas bien?
--------------------------------------------------
English: Machine learning models require large datasets to achieve high accuracy.
French: Les modèles d'apprentissage automatique nécessitent de gros ensembles de données pour obtenir une grande précision.
--------------------------------------------------
```

```
"""
STEP 8: Conclusion

This section summarizes the overall findings of the experiment.
"""

conclusion = """
The experiment demonstrates that Sequence-to-Sequence models are effective
for machine translation tasks. Hugging Face pipelines simplify implementation
by providing easy access to pretrained models. The translations are accurate
for simple sentences, while complex sentences may introduce minor errors.
Overall, the system is efficient, fast, and useful for real-world applications,
though it is not perfect.
"""
```

```
# Print conclusion
print(conclusion)
```

```
The experiment demonstrates that Sequence-to-Sequence models are effective
for machine translation tasks. Hugging Face pipelines simplify implementation
by providing easy access to pretrained models. The translations are accurate
for simple sentences, while complex sentences may introduce minor errors.
Overall, the system is efficient, fast, and useful for real-world applications,
though it is not perfect.
```